Graph Theory and Python Exercises

Algoritmos Avançados de Bioinformática

April 23, 2026

Section 1

Fundamental Graph Concepts

Basic Definitions

- **Vertex (Node):** The fundamental unit of a graph (e.g., a person in a social network).
- **Edge (Arc):** A connection between two vertices.
- **Undirected Graph:** Edges are bidirectional.
- **Directed Graph (Digraph):** Edges have a specific direction.

Neighborhoods

- **Adjacency:** Two vertices are "adjacent" if there is an edge connecting them.
- **Successor:** In a directed edge $U \to V$, V is the successor of U.
- **Predecessor:** In a directed edge $U \to V$, U is the predecessor of V.

Paths and Connectivity

- **Path:** A sequence of edges joining a sequence of distinct vertices.
- **Cycle:** A path that starts and ends at the same vertex.
- **Connectivity:** - *Undirected:* Path exists between every pair.
  - *Directed (Strong):* Path exists in both directions for all pairs.
- **Reachability:** V is reachable from U if a path exists from U to V.

Section 2

Graph Representations

Adjacency Matrix

An **Adjacency Matrix** is a 2D array of size $V \times V$ where V is the number of vertices.

- If there is an edge from vertex i to vertex j, the cell $A[i][j]$ is set to **1** (or the edge weight).
- Otherwise, the cell is set to **0**.
- **Pros:** Constant time $O(1)$ to check if an edge exists.
- **Cons:** Space complexity is $O(V^2)$, which is inefficient for "sparse" graphs (graphs with few edges).

Adjacency List

An **Adjacency List** represents a graph as a collection of linked lists or arrays.

- Each vertex has a list of all the other vertices it is connected to.
- In Python, this is usually implemented as a **Dictionary**: `G = {0: [1, 2], 1: [2], 2: [0, 3], 3: []}`
- **Pros:** Space efficient $O(V + E)$; faster to iterate over neighbors.
- **Cons:** Checking if a specific edge (u, v) exists takes $O(\text{degree}(u))$ time.

Comparison: Matrix vs. List

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Edge Lookup	$O(1)$	$O(\text{degree}(V))$
Add Vertex	$O(V^2)$	$O(1)$
Iterate Neighbors	$O(V)$	$O(\text{degree}(V))$
Best For...	Dense Graphs	Sparse Graphs

Implementation Example (Python)

```python
# Adjacency List (Common in Python)
adj_list = {
    'A': ['B', 'C'],
    'B': ['C'],
    'C': ['A']
}

# Adjacency Matrix (using nested lists)
# A=0, B=1, C=2
adj_matrix = [
    [0, 1, 1], # A connects to B, C
    [0, 0, 1], # B connects to C
    [1, 0, 0]  # C connects to A
]
```

Section 3

Structural Concepts

Degree and Neighborhoods

- **In-Degree:** The number of edges entering a vertex.
- **Out-Degree:** The number of edges leaving a vertex.
- **Isolated Vertex:** A vertex with a degree of 0.
- **Regular Graph:** A graph where every vertex has the same degree.
- **Sources and Sinks:**
  - A **Source** has an in-degree of 0.
  - A **Sink** has an out-degree of 0.

Subgraphs and Complements

- **Subgraph:** A graph formed from a subset of the vertices and edges of G.
- **Complement ($\bar{G}$):** A graph with the same vertices, but edges exist only where they *do not* exist in G.
- **Transpose Graph:** Created by reversing the direction of every edge in a directed graph.
- **Isomorphism:** Two graphs are isomorphic if they contain the same number of vertices connected in the same way, regardless of their visual "layout."

Section 4

Connectivity and Paths

Types of Paths

- **Simple Path:** A path with no repeated vertices.
- **Shortest Path:** The path between two nodes with the minimum number of edges (or minimum total weight).
- **Eulerian Path:** A path that visits every **edge** exactly once.
- **Hamiltonian Path:** A path that visits every **vertex** exactly once.

Components and Cut-points

- **Connected Components:** Subsets of vertices where every vertex is reachable from any other in the same subset.
- **Articulation Point:** A vertex whose removal increases the number of connected components.
- **Bridge:** An edge whose removal disconnects the graph.
- **Bipartite Graph:** A graph whose vertices can be divided into two independent sets U and V such that every edge connects a vertex in U to one in V.

Section 5

Trees and Cycles

Tree Properties

A **Tree** is a special type of graph that is:

1. **Connected:** Every node can reach every other node.
2. **Acyclic:** It contains no cycles.
3. **Minimally Connected:** Removing any edge disconnects the graph.

Here are some definitions about trees:

- **Leaf Node:** A node with degree 1 (in a tree).
- **Spanning Tree:** A subgraph that is a tree and includes all vertices of G.
- **Minimum Spanning Tree (MST):** A spanning tree where the sum of edge weights is minimized.

Directed Acyclic Graphs (DAGs)

- **DAG:** A directed graph with no cycles.
- **Topological Sort:** A linear ordering of vertices such that for every directed edge uv, vertex u comes before v.
- **Reachability:** Often calculated using the **Transitive Closure** of the graph.

Section 6

Advanced Metrics

Centrality and Clustering

- **Clustering Coefficient:** Measures how close a vertex's neighbors are to being a "clique" (fully connected).
- **Betweenness Centrality:** Measures how often a node acts as a bridge along the shortest paths between other nodes.
- **Closeness Centrality:** The reciprocal of the sum of the shortest path distances from a node to all other nodes.
- **PageRank:** An algorithm used to rank the importance of nodes based on the quantity and quality of links to them.

Network Flow

- **Capacity:** The maximum amount of "flow" an edge can handle.
- **Max Flow:** The maximum amount of data/fluid that can pass from a Source to a Sink.
- **Min-Cut:** The minimum total weight of edges that, if removed, would completely separate the Source from the Sink.
- **Max-Flow Min-Cut Theorem:** The value of the max flow is equal to the capacity of the min-cut.

Section 7

Graph Traversal Algorithms

Breadth-First Search (BFS)

BFS explores the graph layer by layer, visiting all neighbors at the current distance before moving further out.

- **Mechanism:** Uses a **Queue** (FIFO).
- **Complexity:** $O(V + E)$.
- **Key Use Cases:**
  - Finding the shortest path in **unweighted** graphs.
  - Testing if a graph is bipartite.
  - Finding all nodes within a specific distance k.

Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking.

- **Mechanism:** Uses a **Stack** (LIFO) or **Recursion**.
- **Complexity:** $O(V + E)$.
- **Key Use Cases:**
  - Detecting cycles in directed and undirected graphs.
  - Topological sorting of a DAG.
  - Finding Strongly Connected Components (SCCs).

Graph Example

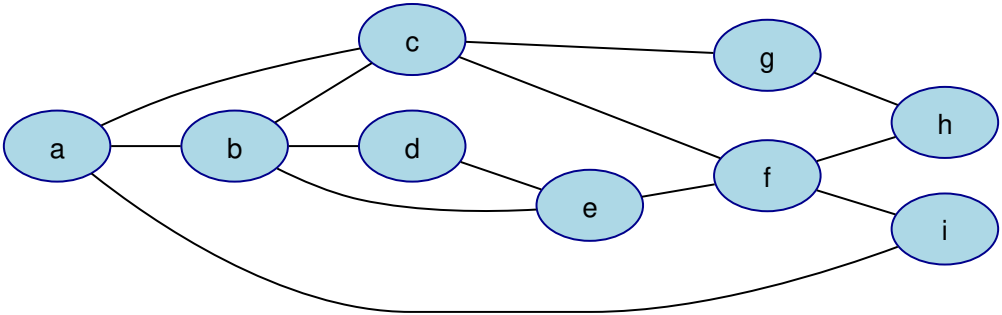

Figure 1: Graph

BFS Example

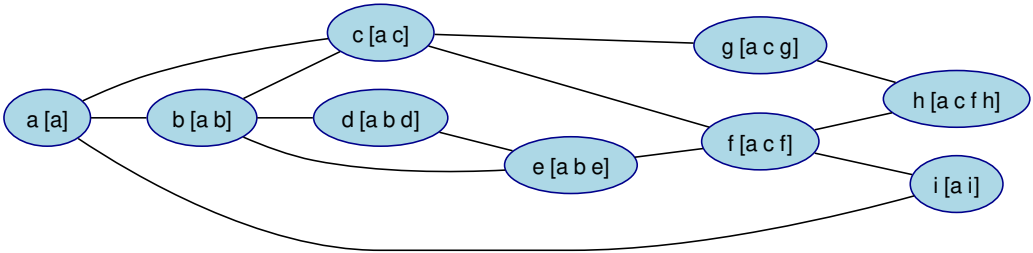

Figure 2: BFS

DFS Example

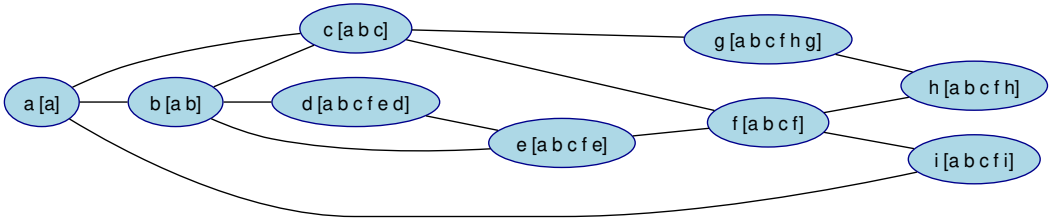

Figure 3: DFS

Section 8

Shortest Path Algorithms

Dijkstra's Algorithm

Dijkstra finds the shortest path from a single source vertex to all other vertices in a **weighted** graph.

- **Requirement:** Edge weights must be **non-negative**.
- **Mechanism:** Uses a **Priority Queue** (Min-Heap) to always expand the node with the smallest known distance.
- **Complexity:** $O((V + E) \log V)$ with a binary heap.

BFS vs. DFS vs. Dijkstra

Algorithm	Data Structure	Best For...
BFS	Queue	Shortest path (unweighted)
DFS	Stack/Recursion	Connectivity & Cycles
Dijkstra	Priority Queue	Shortest path (weighted)

Section 9

Specialized Graphs: Overlap Graphs

Concept of Overlap

An **Overlap Graph** is a directed graph used to represent the relationship between strings.

- **Vertices:** Each vertex represents a string (or a DNA "read").
- **Edges:** A directed edge exists from vertex S_i to S_j if a **suffix** of S_i matches a **prefix** of S_j.
- **Weight:** Often, the edge weight represents the length of the overlapping sequence.

Formal Definition

Given a set of strings $S = \{s_1, s_2, \ldots, s_n\}$ and a minimum overlap threshold k:

- There is an edge $s_i \to s_j$ if:

$$\text{suffix}(s_i, l) = \text{prefix}(s_j, l) \text{ where } l \geq k$$

- The goal in many applications (like genome assembly) is to find a **Hamiltonian Path** that visits each string once to reconstruct the original sequence.

Example of Overlap Graph

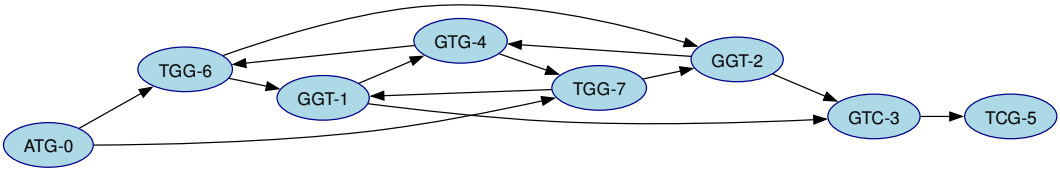

Figure 4: Overlap graph for kmers ATG GGT GGT GTC GTG TCG TGG TGG

Section 10

Specialized Graphs: De Bruijn Graphs

The De Bruijn Concept

A **De Bruijn Graph** is a directed graph used to represent overlaps between symbols in a sequence. It is defined by a set of strings of length k (called **k-mers**).

- **Vertices:** All unique prefixes and suffixes of length $k-1$ found in the k-mers.
- **Edges:** Each k-mer represents a directed edge connecting its $(k-1)$ prefix to its $(k-1)$ suffix.
- **Efficiency:** Unlike Overlap graphs, De Bruijn graphs handle redundant data much better, as identical overlaps collapse into the same nodes and edges.

De Bruijn vs. Overlap Graphs

Feature	Overlap Graph	De Bruijn Graph
Vertex represents	Full read/string	$(k-1)$-mer
Edge represents	Overlap	Full k-mer
Goal	Hamiltonian Path	Eulerian Path
Scaling	Hard (Quadratic edges)	Easy (Linear with k-mers)

[Image comparing Overlap graph and De Bruijn graph]

De Bruijn and the Eulerian Path

The most powerful property of a De Bruijn graph is that reconstructing the original sequence becomes an **Eulerian Path** problem (finding a path that uses every edge exactly once).

- As seen in **Exercise 63**, finding an Eulerian circuit is computationally efficient ($O(E)$).
- This is much faster than the Hamiltonian path required by overlap graphs, which is an NP-complete problem.

De Bruijn Graph Example

```
ATG
 TGG
  GGT
   GTG
    TGG
     GGT
      GTC
       CCG
```

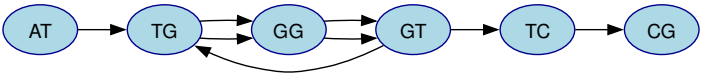

Figure 5: De Bruijn graph example of kmers ATG TGG TGG GGT GGT GTC TCG GTG

De Bruijn Graph Example

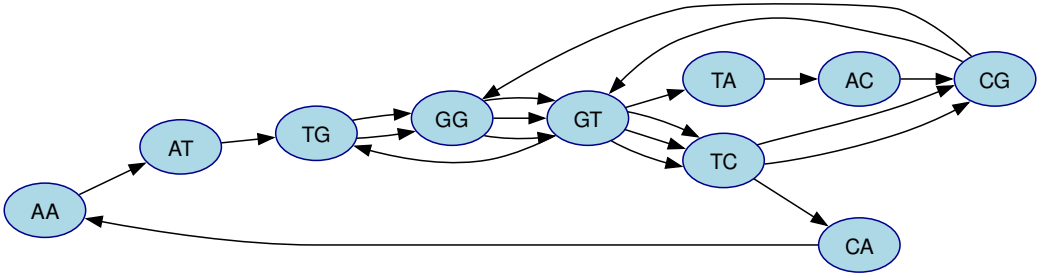

Figure 6: Graph for kmers of size 3 for sequence GTCGTACGGTCAATGGTGGTCG

De Bruijn Graph Example

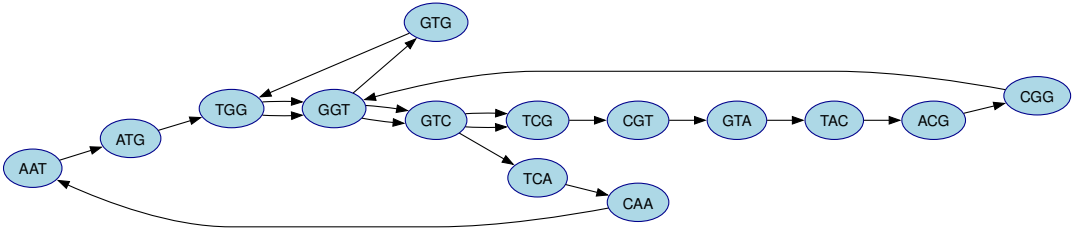

Figure 7: GGraph for kmers of size 4 for sequence TCGTACGGTCAATGGTGGTCG

De Bruijn Graph Example

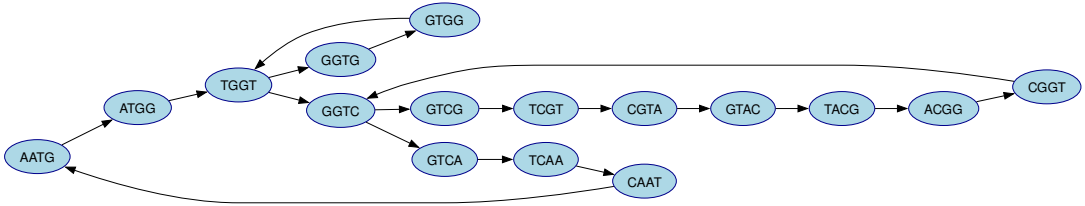

Figure 8: GGraph for kmers of size 5 for sequence TCGTACGGTCAATGGTGGTCG

Section 11

Reconstructing Sequences

From Graph to Sequence

Once the De Bruijn graph is built, the original sequence is reconstructed by finding an **Eulerian Path** (a path that visits every edge exactly once).

- In a De Bruijn graph, every edge represents a k-mer.
- By traversing the edges in the correct order, we "stitch" the sequence back together.
- For a sequence to exist, the graph must be connected and have:
  - All vertices with in-degree = out-degree (Eulerian Circuit).
  - OR exactly two vertices where $|\text{in-degree} - \text{out-degree}| = 1$ (Eulerian Path).

Fleury's Algorithm

Fleury's algorithm is a "greedy" approach with one strict rule: **Don't burn your bridges.**

1. **Start** at a vertex with out-degree $>$ in-degree (if it exists) or any vertex.
2. **Choose an edge** to traverse.
3. **The Rule:** Only cross a **bridge** (an edge whose removal disconnects the graph) if there are no other edges available.
4. **Remove** the edge from the graph after crossing it.
5. **Repeat** until all edges are traversed.

Bridge Detection in Fleury's

To implement Fleury's algorithm effectively in Python, the function must check if an edge is critical.

- An edge (u, v) is a bridge if removing it reduces the total number of reachable vertices from u.
- **Algorithm logic:**
  1. Count reachable nodes from u.
  2. Hypothetically remove edge (u, v).
  3. Re-count reachable nodes.
  4. If the count decreased, the edge is a bridge.

Section 12

Biological Network Motifs

What are Network Motifs?

Network Motifs are subgraphs (patterns) that appear in a real network much more frequently than they would in a randomized version of that same network.

- First popularized by Uri Alon in the study of *E. coli* gene regulation.
- **Significance:** In biology, if a pattern is over-represented, it usually implies that the pattern has been selected by evolution to perform a specific functional task.
- These motifs allow for biological "logic" like delays, pulse generation, and oscillations.

Common Motif: Feed-Forward Loop (FFL)

The most common motif in gene regulation networks, consisting of three genes (X, Y, Z) and three edges.

- **Structure:**
  - X regulates Y and Z.
  - Y also regulates Z.
- **Function:**
  - **Coherent FFL:** Can act as a "sign-sensitive delay" (filtering out short-term noise).
  - **Incoherent FFL:** Can act as a "pulse generator" or a response accelerator.

Other Important Motifs

- **Single Input Module (SIM):** One regulator X controls a group of target genes $(Z_1, Z_2, \ldots, Z_n)$.
  - *Function:* Coordinates a suite of genes with related functions (e.g., metabolic pathways).
- **Bowtie:** One regulator X that is regulated by genes $U_1, \ldots, U_m$ and regulated genes $L_1, \ldots, L_n$.
- **Bifan:** Two regulators (X_1, X_2) both control two target genes (Z_1, Z_2).
  - *Function:* Combinatorial logic or signal integration.
- **Negative Auto-regulation:** A gene X inhibits its own production.
  - *Function:* Stabilizes expression levels and speeds up response time.

Examples of motifs

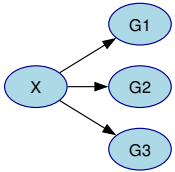

Figure 9: Single Input Module

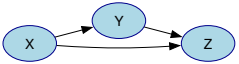

Figure 10: Feed Forward Loop

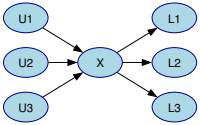

Figure 11: Bowtie

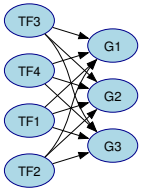

Figure 12: Dense Overlap Regulon

Finding Motifs (The Algorithm)

To identify motifs, we follow these steps:

1. **Count:** Count how many times a specific subgraph appears in the real network G.
2. **Randomize:** Create thousands of "random" versions of G that keep the same degree distribution (number of edges per node).
3. **Compare:** Calculate the **Z-score** or **P-value**.
   - If the count in the real graph is significantly higher than in the random graphs, it is a **Motif**.

Section 13

Exercises

Basic Structure

Create the following functions where G is an adjacency list:

1. `count_vertices(G)`: Total number of vertices.
2. `count_edges(G)`: Total number of edges.
3. `get_successors(G, u)`: List of all successors of u.
4. `get_predecessors(G, u)`: List of all predecessors of u.
5. `is_adjacent(G, u, v)`: True if edge $u \to v$ exists.
6. `get_degree(G, u)`: Number of edges connected to u.
7. `get_in_degree(G, u)`: Incoming edges for u.
8. `get_out_degree(G, u)`: Outgoing edges for u.

Basic Structure

9. `list_isolated_vertices(G)`: Vertices with no edges.
10. `get_all_edges(G)`: List of tuples of all edges.
11. `is_regular(G)`: Check if all vertices have the same degree.
12. `get_sources(G)`: Vertices with in-degree 0.
13. `get_sinks(G)`: Vertices with out-degree 0.
14. `add_vertex(G, u)`: Add a new vertex to the graph.
15. `remove_edge(G, u, v)`: Delete the edge between u and v.
16. `remove_vertex(G, u)`: Delete the vertex u from the graph.

Representations

17. `to_adjacency_matrix(G)`: Convert to 2D array.
18. `from_adjacency_matrix(G, matrix)`: Rebuild G from matrix.
19. `get_complement(G)`: Return the inverse graph.
20. `is_undirected(G)`: Check if every edge (u, v) has (v, u).
21. `to_edge_list(G)`: Convert to list of pairs.
22. `is_sparse(G)`: True if $E \ll V^2$.
23. `transpose_graph(G)`: Reverse all edge directions.
24. `copy_graph(G)`: Create a deep copy.

Modification & Weights

25. `merge_graphs(G, H)`: Combine two graphs.
26. `subgraph(G, vertices)`: Filter G by a list of vertices.
27. `is_isomorphic_simple(G, H)`: Check if small graphs are identical.
28. `get_heaviest_edge(G)`: Find edge with max weight.
29. `normalize_weights(G)`: Scale weights between 0 and 1.
30. `remove_self_loops(G)`: Remove edges where $u = v$.

Connectivity & Paths

91. `has_path(G, u, v)`: Check reachability.
92. `find_any_path(G, u, v)`: Return any valid path.
93. `all_simple_paths(G, u, v)`: Find all paths (no repeats).
94. `find_shortest_path_unweighted(G, u, v)`: Use BFS.
95. `is_connected_undirected(G)`: Connectivity check.
96. `count_connected_components(G)`: Count islands.
97. `get_connected_components(G)`: Return sets of vertices.
98. `find_diameter(G)`: Longest shortest path.
99. `is_reachable_within_k(G, u, v, k)`: Check distance constraint.
100. `get_distance_matrix(G)`: All-pairs shortest paths.

Advanced Connectivity

41. `is_bridge(G, u, v)`: Does removal disconnect G?
42. `find_articulation_points(G)`: Find critical nodes.
43. `is_bipartite(G)`: Can it be 2-colored?
44. `get_eccentricity(G, u)`: Max distance from u.
45. `find_radius(G)`: Minimum eccentricity.
46. `find_center(G)`: Vertices with min eccentricity.
47. `is_strongly_connected(G)`: Directed graph check.
48. `is_weakly_connected(G)`: Undirected version check.
49. `find_path_length(G, path)`: Sum of weights.
50. `find_all_distances_from_source(G, s)`: Dijkstra/BFS dict.

Cycles & Trees

91. `has_cycle_undirected(G)`: Cycle detection.
92. `has_cycle_directed(G)`: Use DFS/Recursion stack.
93. `is_tree(G)`: Connected and acyclic.
94. `is_forest(G)`: Collection of trees.
95. `find_cycle_nodes(G)`: Nodes belonging to cycles.
96. `is_dag(G)`: Directed Acyclic Graph check.
97. `topological_sort(G)`: Linear ordering for DAG.
98. `count_leaf_nodes(G)`: Nodes with degree 1.

Euler & Hamilton

99. `is_eulerian_path_exists(G)`: Degree parity check.
60. `find_eulerian_circuit(G)`: Hierholzer's or Fleury's.
01. `is_hamiltonian_cycle(G, path)`: Verify a cycle.
02. `get_tree_height(G, root)`: Max depth.
03. `list_ancestors(G, u, root)`: Nodes on path to root.
04. `find_lca(G, u, v, root)`: Lowest Common Ancestor.
05. `is_planar_small(G)`: Visual/Euler formula check.
06. `count_triangles(G)`: 3-cycles.
07. `find_clique_size_3(G)`: Complete subgraphs of 3.

Centrality & SCCs

68. `betweenness_centrality(G, u)`: Shortest path bridge score.
69. `closeness_centrality(G, u)`: Distance-based score.
70. `transitive_closure(G)`: All reachability edges.
71. `get_scc_kosaraju(G)`: Two-pass SCC.
72. `check_2_colorability(G)`: Color assignment.
73. `find_maximum_clique(G)`: Largest complete subgraph.

Algorithms & Flow

74. `dijkstra(G, start, end)`: Shortest path.
75. `bellman_ford(G, start)`: Handle negative weights.
76. `floyd_warshall(G)`: Matrix-based paths.
77. `find_negative_cycles(G)`: Detection via Bellman-Ford.
78. `get_clustering_coefficient(G, u)`: Local density.
79. `get_average_clustering(G)`: Global density.

Biological Networks

80. `create_met_react_graph(file)`: Read a file with a list of reactions and create the corresponding metabilite reaction graph.
81. `is_met_react_graph(G)`: Check if the graph is bipartite.
82. `extract_react_met(G)`: Return a tuple with the reactions and metabilites from the corresponding metabilite reaction graph.
83. `create_met_graph(file)`: Read a file with a list of reactions and create the corresponding metabilite graph.
84. `create_react_graph(file)`: Read a file with a list of reactions and create the corresponding reaction graph.
85. `extract_react_graph(G)`: Extract the reaction graph for the corresponding metabilite reaction graph.
86. `extract_met_graph(G)`: Extract the metabolite graph for the corresponding metabilite reaction graph.

Assembly

87. `overlap_graph(kmers)`: Return overlap graph.
88. `is_reconstruction(kmers)`: Is the list kmers a path over an overlap graph?
89. `reconstruction_overlap_path(kmers)`: Reconstruct the sequence from the path.
90. `debruijn_graph(kmers)`: Return De Bruijn graph.
91. `reconstruct_sequence(kmers)`: Reconstruct the original sequence from the list of kmers.
92. `get_all_contigs(kmers)`: Get all contigs from the kmers.